Credify: Decentralized Multi-Lender Credit Protocol

Comprehensive Architectural Blueprint, Threat Analysis, and Blockchain Fundamentals

Academic Compendium & Technical Reference | Date: 2026-10-15

Abstract

This document provides an exhaustive, academic, and architectural exposition of **Credify**, a decentralized multi-lender commercial credit protocol. Credify introduces autonomous smart contract escrow pools, on-chain capital expenditure controls through allowlisted supplier drawdowns, a mathematically proven non-locking pro-rata repayment waterfall, EIP-1167 minimal proxy cloning, and volume-weighted counterparty-diverse credit reputation. We detail the protocol's game-theoretic trust model, present an audited seven-vector threat matrix with explicit reentrancy mitigations, explore the reasons behind avoiding external databases, and provide foundational explanations of core blockchain concepts underpinning the system.

Contents

1 Protocol Overview & Core Architectural Theory . . . 2
1.1 The Custodial Problem in Traditional P2P Lending . . . 2
1.2 Autonomous Syndicate Escrow (`LoanPool.sol`) . . . 2
1.3 All-or-Nothing Funding Window & Pull-Payment Safety . . . 2
2 Controlled Disbursements via Allowlisted Suppliers (Anti-Diversion) . . . 2
2.1 The Capital Diversion Hazard . . . 2
2.2 Zero Borrower Custody & Programmatic Spend Limits . . . 2
3 Pro-Rata Waterfall & Mathematical Derivation . . . 3
3.1 The "Locked Funds" Defect in Naive Protocols (ADR-001) . . . 3
3.2 The Credify Non-Locking Waterfall Formula . . . 3
3.3 Mathematical Simulation Walkthrough . . . 3
4 Academic Stretch Architecture: EIP-1167 Proxies & Sybil Defense . . . 3
4.1 EIP-1167 Minimal Proxy Clones (Gas Economics) . . . 3
4.2 Sybil-Resistant Weighted Reputation Scoring . . . 4
5 Formal Threat Analysis & Security Architecture . . . 4
5.1 Deep Dive: Reentrancy Protection . . . 4
5.2 The 7 Formal Threat Vectors Matrix . . . 4
6 Database Architecture & Blockchain CQRS . . . 5
6.1 Why No External Database (PostgreSQL / MongoDB) Is Used . . . 5
6.2 Why Blockchain Databases (e.g. BigchainDB) Are Inapplicable . . . 5
7 Complete Smart Contract Catalog . . . 5
8 Core Blockchain Concepts Explained . . . 6
8.1 The EVM, Opcodes & Storage Slots . . . 6
8.2 Gas, Block Gas Limits & EIP-1559 Base Fee . . . 6
8.3 Events, Bloom Filters & Topics . . . 6
8.4 Account Nonces & Cryptographic Signatures . . . 6
8.5 EIP-1167 `DELEGATECALL` Mechanics . . . 6

—

1 Protocol Overview & Core Architectural Theory

1.1 The Custodial Problem in Traditional P2P Lending

In traditional finance and legacy Web2 peer-to-peer (P2P) lending platforms (e.g., LendingClub, Prosper), loan syndication relies on a centralized intermediary. The operator pools lender funds into omnibus bank accounts, manually processes supplier disbursements, and reconciles monthly interest repayments.

This legacy structure introduces three fatal flaws:

1. **Custodial & Counterparty Risk:** Operator insolvency, freezing of bank accounts, or embezzlement traps investor funds.
2. **Information Asymmetry:** Syndicate lenders cannot independently verify whether borrowed principal was spent on legitimate commercial activities or diverted into speculative trading.
3. **High Friction & Intermediation Fees:** Manual auditing and custodial legal overhead significantly diminish lender net yields.

1.2 Autonomous Syndicate Escrow (`LoanPool.sol`)

Credify replaces human custodians with isolated, sovereign smart contract vaults deployed on the Ethereum Virtual Machine (EVM). Neither the borrower nor protocol administrators hold private keys to withdraw pooled funds. Funds can exit the contract vault strictly via two programmatic channels:

- **Direct Supplier Disbursement:** Paying approved invoices directly to pre-vetted supplier wallets (`spend()`).
- **Pro-Rata Lender Withdrawals:** Syndicate lenders pulling their proportional share of repayments and unspent principal (`claimRepayment()`).

1.3 All-or-Nothing Funding Window & Pull-Payment Safety

Lenders face a coordination dilemma when contributing asynchronously: **What if I deposit funds, but other lenders fail to join, leaving my capital trapped in a half-funded loan?**

- **All-or-Nothing Enforcement:** Every loan pool establishes an immutable `fundingDeadline`. If cumulative contributions do not reach 100% of `targetWei` prior to deadline expiration, the pool cannot activate.
- **Guaranteed Principal Refunds:** Any participant can invoke `cancelUnfunded()`, transitioning the contract to `Cancelled` and immediately unlocking 100% principal refunds via `claimFundingRefund()`.
- **Pull-Payment Pattern:** In naive contracts, loans iterate through lender arrays to "push" ETH. If any recipient is a contract that rejects transfers, the transaction runs out of gas, permanently freezing the contract. Credify uses the **Pull-Payment Pattern**: each lender independently pulls their own claim, eliminating gas exhaustion vulnerabilities.

—

2 Controlled Disbursements via Allowlisted Suppliers (Anti-Diversion)

2.1 The Capital Diversion Hazard

In unsecured commercial lending, the primary driver of default is **capital diversion**: a corporate borrower requests credit for operational equipment or raw inventory, but instead diverts the cash into speculative cryptocurrency trading, personal luxury assets, or capital flight.

2.2 Zero Borrower Custody & Programmatic Spend Limits

Credify solves this through smart-contract enforced spending controls:

1. **Zero Direct Cash-Out:** The `LoanPool` smart contract contains **no function** permitting the borrower to withdraw or transfer pooled ETH to their own address.
2. **Allowlisted Supplier Directory:** At agreement creation, the borrower must specify up to 10 pre-vetted supplier addresses (`merchants`), each bounded by an immutable per-merchant cap (`merchantCapWei`) and an overall facility limit (`maxSpendWei`).
3. **Atomic Verification on Spend:** When invoking `spend(merchant, amount, category)`, the contract checks:

$$\text{totalSpent} + \text{amount} \leq \text{maxSpendWei}$$

$$\text{merchantSpentWei}[\text{merchant}] + \text{amount} \leq \text{merchantCapWei}[\text{merchant}]$$

Funds transfer atomically from the pool directly to the supplier's address in the same transaction. The borrower never touches the funds.

—

3 Pro-Rata Waterfall & Mathematical Derivation

3.1 The "Locked Funds" Defect in Naive Protocols (ADR-001)

In conventional multi-lender prototypes, lender dividend claims are computed solely against total borrower repayments:

$$\text{Claimable}_{\text{naive}} = \frac{\text{Contributed}_{\text{lender}} \times \text{totalRepaid}}{\text{totalContributed}} - \text{Claimed}_{\text{lender}}$$

The Fatal Defect: Suppose a syndicate raises 10 ETH, but the borrower only spends 6 ETH on inventory. At maturity, the borrower repays 6.48 ETH (6 ETH principal + 8% interest). Lenders claim against the 6.48 ETH, but the remaining **4.00 ETH of unspent principal remains permanently trapped in the contract vault forever**!

3.2 The Credify Non-Locking Waterfall Formula

Credify resolves this vulnerability by introducing the dynamic `distributable()` view function in `LoanPool.sol`:

$$\text{distributable}() = \begin{cases} \text{totalRepaid} + (\text{totalContributed} - \text{totalSpent}) & \text{if status} \in \{\text{Repaid}, \text{Defaulted}\} \\ \text{totalRepaid} & \text{if status} = \text{Active} \end{cases}$$

When a lender invokes `claimRepayment()`, their cumulative entitlement is computed as:

$$\text{Entitled}_{\text{lender}} = \left\lfloor \frac{\text{distributable}() \times \text{Contributed}_{\text{lender}}}{\text{totalContributed}} \right\rfloor$$

$$\text{Payout} = \text{Entitled}_{\text{lender}} - \text{Claimed}_{\text{lender}}$$

3.3 Mathematical Simulation Walkthrough

Consider the live simulation verified in the protocol test suite:

- Total Capital Contributed: 10.00 ETH (Lender A: 5 ETH [50%], Lender B: 3 ETH [30%], Lender C: 2 ETH [20%]).
- Disbursed to Suppliers (`totalSpent`): 6.00 ETH.
- Unspent Principal Left in Vault: $10.00 - 6.00 = 4.00$ ETH.
- Repaid by Borrower: 6.48 ETH (6 ETH principal + 8% interest on spent capital).

$$\text{distributable}() = 6.48 + (10.00 - 6.00) = \mathbf{10.48\ ETH}$$

Individual Pro-Rata Entitlements:

- **Lender A (50%):** $\frac{10.48 \times 5.00}{10.00} = \mathbf{5.2400\ ETH}$ $(+4.80\%$ net return$)$
- **Lender B (30%):** $\frac{10.48 \times 3.00}{10.00} = \mathbf{3.1440\ ETH}$ $(+4.80\%$ net return$)$
- **Lender C (20%):** $\frac{10.48 \times 2.00}{10.00} = \mathbf{2.0960\ ETH}$ $(+4.80\%$ net return$)$

Total distributed: $5.2400 + 3.1440 + 2.0960 = \mathbf{10.48\ ETH}$. The vault balance returns to **exactly 0 wei**. Zero capital is trapped.

—

4 Academic Stretch Architecture: EIP-1167 Proxies & Sybil Defense

4.1 EIP-1167 Minimal Proxy Clones (Gas Economics)

Deploying full loan contract bytecode (`new LoanPool(...)`) writes ≈ 11.2 KB of runtime bytecode into global state on every loan:

- Standard `new LoanPool` Deployment: **2,787,646 gas**.
- EIP-1167 Clone Factory (`Clones.clone`): **580,339 gas**.
- **Net Gas Savings: −79.18% (saving >2.2 Million gas per pool)**.

The 45-byte runtime proxy bytecode forwards calls to a single master implementation (`LoanPoolCloneable`) via `DELEGATECALL`, executing logic directly within the proxy's isolated storage space.

4.2 Sybil-Resistant Weighted Reputation Scoring

Linear scoring ($+8$ on repayment, -20 on default) allows **wash-loan farming**: an attacker creates and repays 7 self-funded micro-loans (0.001 ETH) to artificially reach a 100 score, then takes out a 50 ETH loan and defaults.

Credify's `WeightedReputationRegistry.sol` closes this loophole using two cryptographic constraints:

1. **Volume Tier Scaling:** Loans < 0.1 ETH yield only $+1$ pt; loans ≥ 5 ETH award full $+8$ pts.
2. **Counterparty Diversity Hard-Caps:**

$$\text{Score}_{\max(N_{\text{unique}})} = \begin{cases} \mathbf{65} & \text{if } N_{\text{unique}} < 2 \quad \text{(Wash-loan hard cap)} \\ \mathbf{80} & \text{if } N_{\text{unique}} < 4 \quad \text{(Partial syndicate diversity)} \\ \mathbf{100} & \text{if } N_{\text{unique}} \geq 4 \quad \text{(Tier-1 multi-lender credit unlocked)} \end{cases}$$

—

5 Formal Threat Analysis & Security Architecture

5.1 Deep Dive: Reentrancy Protection

Reentrancy occurs when an external ETH transfer hands execution control to a recipient contract whose fallback function immediately calls back into the victim contract before state updates finalize. Credify enforces a **dual-layer defense**:

1. **Checks-Effects-Interactions (CEI):** In `claimRepayment()` and `spend()`, internal storage mappings (`claimed[msg.sender]`, `totalSpent`) are modified **prior** to executing low-level transfers.
2. **OpenZeppelin `nonReentrant` Mutex Guard:** A mutex storage flag (`_ENTERED`) locks functions during execution, reverting nested calls.

Formal security testing validated this defense against dedicated exploit contracts (`MaliciousMerchant` and `MaliciousLender`).

5.2 The 7 Formal Threat Vectors Matrix

Threat Vector	Attack Scenario	Smart Contract Mitigation
1. Sybil Borrower	Disposable addresses borrow and default.	`KYCRegistry` salted attestation required; raw funds locked to supplier invoices.
2. Supplier Collusion	Borrower routes cash to fake supplier address.	Suppliers verified by `VERIFIER_ROLE`; max 10 merchants; immutable per-merchant spend caps.
3. Reentrancy	Malicious fallback drains funds recursively.	OpenZeppelin `nonReentrant` + Checks-Effects-Interactions (CEI) state sequencing.
4. Default Griefing	Syndicate lenders liquidate borrower early.	`voteDefault()` locked until maturity + 3-day grace period; requires >50.01% capital quorum.
5. Trapped Funds	Unspent capital frozen in contract vault.	Dynamic `distributable()` formula (ADR-001) returns unspent principal pro-rata.

6. Underfunded Lockup	Loan raises partial funds and freezes.	`fundingDeadline` unlocks 1-click `cancelUnfunded()` and 100% principal refunds.
7. Governance Hijack	Borrower or unverified actor buys vote weight.	`contribute()` requires `Role.Lender`; borrower explicitly banned from contributing.

—

6 Database Architecture & Blockchain CQRS

6.1 Why No External Database (PostgreSQL / MongoDB) Is Used

1. **Single Source of Truth:** In DeFi, keeping loan state in an external database creates a "split-brain" vulnerability where off-chain records diverge from smart contract truth.
2. **Event-Sourced Read Projections (CQRS):** Smart contracts handle state-mutating writes. The Fastify API indexer continuously polls block event receipts (`getLogs`) and writes atomic projections to JSON files (`data/projection.json`).
3. **Zero-Friction Portability:** Eliminates Docker, SQL migrations, and credential management, allowing instant local execution.

6.2 Why Blockchain Databases (e.g. BigchainDB) Are Inapplicable

BigchainDB and decentralized document stores lack Turing-complete EVM execution, cannot custody native ETH or ERC-20 assets, lack EIP-1167 proxy opcode support, and are incompatible with MetaMask and standard Web3 wallet tooling.

—

7 Complete Smart Contract Catalog

- **`LoanPool.sol`:** Core escrow vault.
  - `contribute() payable`: Syndicate lenders deposit capital toward `targetWei`.
  - `cancelUnfunded()`: Cancels loan if underfunded at deadline.
  - `claimFundingRefund()`: Refunds 100% of principal upon cancellation.
  - `spend(merchant, amount, category)`: Disburses funds directly to allowlisted suppliers.
  - `repay() payable`: Borrower repays debt service; awards $+8$ reputation points on completion.
  - `distributable() view`: Computes total claimable funds including unspent principal.
  - `claimRepayment()`: Lenders pull pro-rata repayments and unspent principal.
  - `voteDefault()`: Lenders cast capital-weighted votes after maturity + grace period.
- **`LoanFactory.sol`:** Validates terms, creates pools, authorizes them in `ReputationRegistry`.
  - `createLoan(terms)`: Validates duration, APR, merchant caps, deploys new pool.
  - `pause() / unpause()`: Emergency circuit breaker.
- **`KYCRegistry.sol`:** Salted cryptographic role-based access control.
  - `setVerification(account, role, expiresAt, hash)`: Verifier attests applicant.
  - `revokeVerification(account)`: Revokes credentials.
  - `isVerified(account, role) view`: Evaluates validity and expiration timestamp.
- **`ReputationRegistry.sol`:** Deterministic credit scoring (Base 50, $+8$ repayment, -20 default, bounded $[0, 100]$).
- **`MockUSDC.sol`:** 6-decimal ERC-20 stablecoin with public faucet `mint()`.

—

8 Core Blockchain Concepts Explained

8.1 The EVM, Opcodes & Storage Slots

The EVM is a quasi-Turing-complete, stack-based state machine. Contract state resides in 2^{256} persistent 32-byte **Storage Slots**. Writing a storage slot (`SSTORE`) costs 20,000 gas, whereas in-memory operations cost 3 gas. Mappings calculate slot keys as $\mathrm{keccak256}(\mathrm{key} \parallel \mathrm{slotIndex})$.

8.2 Gas, Block Gas Limits & EIP-1559 Base Fee

Gas quantifies the computational complexity of transactions. The **Block Gas Limit** (e.g. 60,000,000 in Hardhat, 30,000,000 on Mainnet) protects nodes from infinite loops (the Halting Problem), prevents network centralization, and caps block propagation latency. Under EIP-1559, transactions pay a dynamically adjusted Base Fee (which is burned) plus a Priority Tip to validators.

8.3 Events, Bloom Filters & Topics

Solidity events write data to block receipts using `LOG` opcodes at a fraction of storage costs (≈ 375 gas). Indexed parameters (Topics) are hashed into 2048-bit Bloom filters, allowing external indexers to query millions of blocks in constant time without executing bytecode.

8.4 Account Nonces & Cryptographic Signatures

Every Ethereum transaction is signed with an elliptic-curve private key (ECDSA on secp256k1), yielding (v, r, s). The account **Nonce** is a sequential counter preventing replay attacks by guaranteeing that each signed transaction can only be mined once.

8.5 EIP-1167 `DELEGATECALL` Mechanics

Unlike standard `CALL`, `DELEGATECALL` executes the master contract's bytecode within the caller's context, preserving `msg.sender`, `msg.value`, and writing directly to the proxy's storage slots.